Aplicaciones Móviles

GR19

2026-A

Taller: Arquitectura y Evolución del Desarrollo Móvil

Integrantes:

- León Sebastián
- Salazar Gregory
- Yunga Mateo

Fecha de Entrega: 14-04-2026

Parte 1: El Viaje Arquitectónico del Desarrollo Móvil

- **Desarrollo Nativo Puro** Es el estándar de máximo rendimiento y fidelidad al usar lenguajes oficiales (Swift/Kotlin). Al no tener intermediarios, ofrece una fluidez total y acceso inmediato al hardware, aunque a un alto costo operativo por la necesidad de duplicar equipos y código para cada sistema.
- **La Era del WebView / Híbridos** Buscó la eficiencia técnica "envolviendo" páginas web (HTML/JS) en apps descargables. Aunque facilitó el desarrollo multiplataforma, sacrificó la experiencia de usuario con un rendimiento pobre, alto consumo de memoria y una respuesta táctil que nunca se sintió realmente nativa.
- **Puente Nativo-JavaScript (React Native)** Logró un equilibrio inteligente: lógica en JavaScript con una interfaz de componentes nativos reales. Ofrece una apariencia auténtica compartiendo gran parte del código, pero su dependencia de un "puente" de comunicación puede generar cuellos de botella en procesos de datos masivos o animaciones muy complejas.
- **Motor de Renderizado Propio (Flutter)** Propuesta de Google que elimina los componentes del sistema para dibujar cada píxel directamente con su propio motor gráfico. Esto garantiza un rendimiento excepcional y una estética idéntica en cualquier dispositivo, a cambio de apps más pesadas y el esfuerzo de imitar manualmente los gestos nativos.
- **El Futuro: Foldables, AR y Wearables** El desarrollo actual trasciende la pantalla plana hacia entornos espaciales y multidispositivo. Se prioriza la adaptabilidad para pantallas plegables, el renderizado 3D para realidad aumentada y la eficiencia extrema en dispositivos vestibles, gestionando datos que fluyen en tiempo real entre el hardware y la nube.

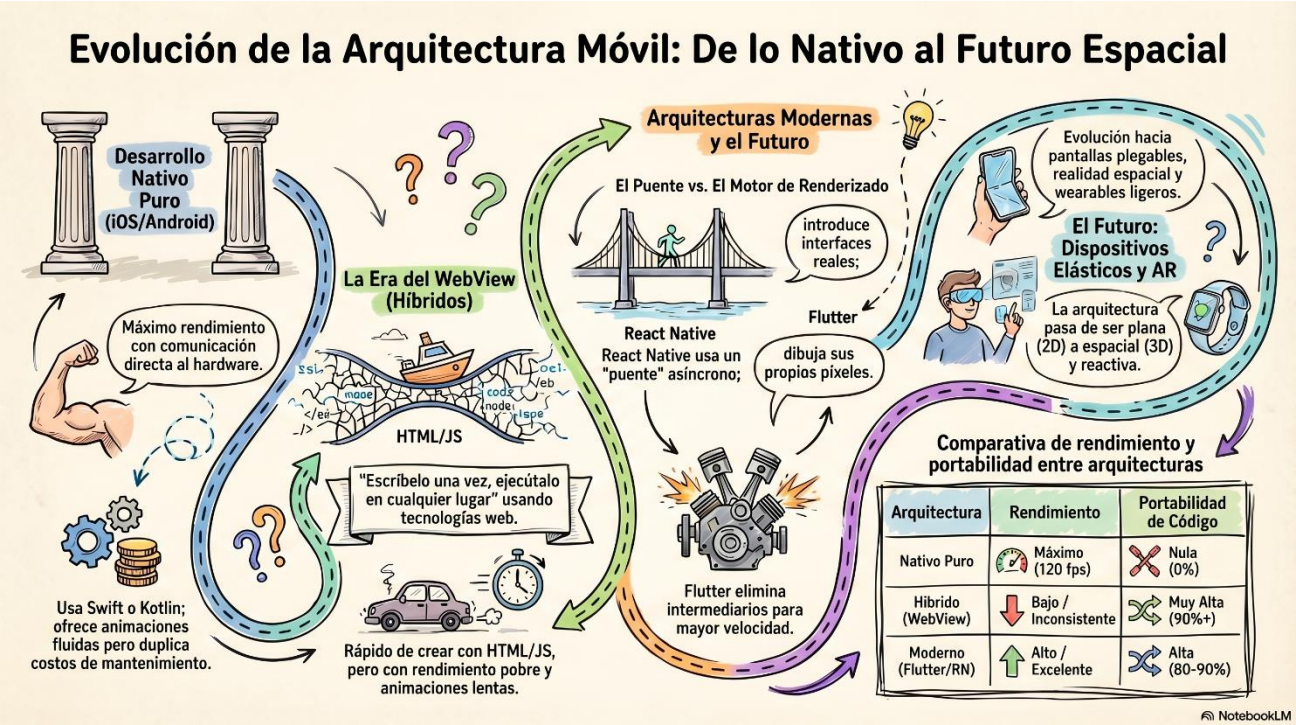

Ilustración 1. Evolución de la arquitectura móvil. Google 2026.

Parte 2: Taller Práctico - Análisis Crítico de Aplicaciones

- **Caso de Estudio A:** Shazam

Shazam nació en 2002 como un servicio de reconocimiento musical accesible por llamada telefónica, años antes de que existieran los smartphones. Con la llegada del iPhone en 2007, Shazam se convirtió en una de las primeras aplicaciones de la App Store y redefinió completamente su modelo. A diferencia de una app de reproducción musical, Shazam es un sistema de identificación acústica en tiempo real cuyo núcleo es un algoritmo de fingerprinting de audio. Cuando un usuario activa la app, el dispositivo captura una muestra de audio crudo, aplica una Transformada Rápida de Fourier para convertirla en un espectrograma de frecuencias, y compara ese patrón contra una base de datos de más de 11 millones de huellas digitales almacenadas en servidores remotos, todo en menos de 5 segundos. En 2018, Apple adquirió Shazam por aproximadamente 400 millones de dólares, integrándolo profundamente en el ecosistema iOS.

- **Caso de Estudio B:** Waze

Waze nació como un proyecto comunitario de mapeo (FreeMap Israel) y evolucionó hasta convertirse en el principal GPS social del mundo, siendo adquirido por Google en 2013 por más de 1.000 millones de dólares.

A diferencia de un mapa estático, Waze es un sistema vivo que depende de la ingesta masiva de datos en tiempo real (accidentes, tráfico, policía) reportados por millones de nodos (usuarios) en movimiento constante.

- **Debatir, investigar y redactar un reporte crítico respondiendo las siguientes interrogantes. No buscar respuestas "correctas", busquen respuestas fundamentadas con ingeniería.**

1. **Ingeniería Inversa de la Decisión:**

- **¿Por qué creen que Shazam, una app cuyo núcleo es el reconocimiento de audio en tiempo real mediante procesamiento de señales (FFT/fingerprinting), apostó por desarrollo nativo en lugar de un framework multiplataforma, considerando que hoy funciona en iOS, Android, Mac y Apple Watch?**

Shazam optó por el desarrollo nativo porque su funcionamiento depende críticamente del reconocimiento de audio en tiempo real, lo cual exige un rendimiento extremadamente alto y una latencia mínima.

El desarrollo nativo permite optimizar al máximo el uso de los recursos de hardware del dispositivo a través del uso de APIs como AudioRecord en Android que operan a nivel del sistema operativo.

Factores como precisión, velocidad y eficiencia se logran de mejor manera mediante un enfoque nativo a diferencia del uso de un framework multiplataforma.

- **¿Por qué Waze construyó su motor principal en C++ compartido con interfaces nativas por plataforma en lugar de optar por React Native o Flutter?**

Al tratarse de una aplicación móvil, el hardware es limitado, es por ello que se priorizó el rendimiento con C++. Algunas de las razones que los llevaron a tomar esa decisión fueron:

- Dibujar mapas 3D a 60 FPS con React Native (por su puente) o Flutter generaría mucho *lag*. C++ se salta los intermediarios y habla directo con la tarjeta gráfica (OpenGL/Metal).

- Mientras se ejecuta, la app necesita leer GPS, descargar tráfico y recalcular rutas al mismo tiempo. JavaScript (React Native) sufre inconvenientes porque es de un solo hilo. C++ maneja múltiples procesos pesados de forma óptima.

- Es mucho más eficiente escribir la matemática pesada (rutas, mapas) una sola vez en C++ y poner una interfaz 100% nativa con Swift o Kotlin. De esta manera se reutiliza alrededor del 80% del código sin perder fluidez nativa.

2. **Análisis de Rendimiento y Arquitectura:**

- **¿Qué desafíos arquitectónicos enfrenta Shazam al tener que capturar audio desde el micrófono, aplicar transformadas de Fourier y consultar su base de datos de huellas digitales en menos de 5 segundos, y cómo impacta esto en la elección entre acceso nativo directo al hardware vs. un puente de abstracción?**

Shazam enfrenta desafíos arquitectónicos críticos al tener que procesar audio en tiempo real en menos de 5 segundos. Primero, debe capturar el flujo continuo del micrófono sin perder muestras, lo cual requiere acceso directo al hardware para evitar interrupciones. Segundo, necesita aplicar la Transformada Rápida de Fourier de forma inmediata, ya que es un cálculo intensivo que no puede introducir latencia adicional. Tercero, debe consultar una gran base de datos de huellas digitales, donde la latencia de red ya consume parte del tiempo disponible. En este contexto, el uso de un puente de abstracción (como en frameworks multiplataforma) añade latencia e imprevisibilidad en cada etapa del proceso, lo que puede impedir cumplir con el tiempo requerido, especialmente en dispositivos menos potentes. Por ello, el acceso nativo directo al hardware resulta fundamental para garantizar velocidad, precisión y eficiencia en todo el pipeline.

- **¿Qué cuello de botella (basado en la arquitectura del "puente") creen que tendría que resolver Shazam si estuviera construida con React Native para lograr que el scroll de miles de canciones en el historial sea fluido a 60 FPS en teléfonos de gama baja?**

Shazam real NO usa React Native, pero si lo hiciera, su mayor desafío no sería el historial (eso es manejable), sino el reconocimiento en tiempo real donde la latencia del puente sería fatal.

- **¿Qué estrategias de software utiliza Waze para no agotar la batería y los datos móviles rápidamente en viajes largos?**

Para optimizar drásticamente la batería, Waze implementa **Throttling y Batching**. El throttling reduce de forma inteligente las consultas al GPS en rectas largas (interpolando el

movimiento en pantalla) y solo aumenta la precisión cerca de los desvíos. En paralelo, el batching acumula los datos de telemetría en la memoria del teléfono y los envía a la nube en "paquetes" cada 15 o 20 segundos; esto permite que la antena de red del celular permanezca inactiva la mayor parte del tiempo, ahorrando muchísima energía.

- **¿Cómo logra Waze que tu auto siga moviéndose en el mapa sin *lag* cuando entras a un túnel y pierdes la señal GPS?**

Waze usa Dead Reckoning mediante una arquitectura reactiva que nunca bloquea la pantalla. Una máquina de estados detecta la falla del GPS y pasa a depender de los sensores locales (acelerómetro y giroscopio). Toda la física pesada para calcular tu nueva posición en base a tu última velocidad se ejecuta en un hilo en segundo plano (Background Worker), enviando los resultados al hilo principal para que el auto siga avanzando en el mapa de forma fluida y sin *lag.*

3. **El Desafío de las Nuevas Pantallas:**

- **Si mañana Apple y Samsung deciden que sus dispositivos estrella serán lentes de Realidad Aumentada interactiva, ¿cuál de estas dos tecnologías (React Native o Flutter) tendría más facilidad de adaptación según su arquitectura subyacente, o ambas fallarían?**

Ambas fallarían. La AR requiere renderizado 3D de ultra-baja latencia, pero Flutter solo renderiza 2D (Skia/Impeller) y RN depende de un puente JS que añade latencia.

También LA AR necesita seguimiento de posición y entorno (SLAM) y eso implica acceso directo a ARKit/ARCore a nivel de hardware (Puente que añade latencia). Incluso con plugins, cada frame de AR tiene que cruzar el MethodChannel (el puente de Flutter hacia código nativo), reintroduciendo latencia.

RN queda completamente descartado por su arquitectura de puente (bridge) entre JavaScript y el hilo nativo es inherentemente asíncrona y serializable. Su ciclo de procesamiento para cada frame causaría una latencia acumulada haría que la experiencia AR se sintiera "mareante" y desconectada.

Por estas razones, Apple y Google diseñan sus SDKs de AR (ARKit, ARCore) primero para sus plataformas nativas. Los frameworks cross-platform siempre llegan tarde y con funcionalidades recortadas.

4. **Veredicto Crítico:**

- **Como futuros ingenieros de software: Si estuvieran creando hoy una**

aplicación que requiere interactuar con mapas 3D interactivos, notificaciones de alta frecuencia y debe soportar pantallas plegables... ¿Elegirían Nativo, React Native o Flutter? Fundamenten su respuesta basados estrictamente en la arquitectura de la herramienta, no en gustos personales por un lenguaje.

Nativo (Swift y Kotlin). Los mapas 3D requieren procesar 60 frames por segundo de geometría, texturas y gestos simultáneamente. Flutter no tiene motor 3D nativo (solo 2D via Impeller). React Native añade latencia por su puente, haciendo que rotar un mapa 3D se sienta "pegajoso" en lugar de fluido.

Además, iOS y Android tienen reglas estrictas para ejecución en segundo plano para preservar batería. Los frameworks cross-platform dependen de plugins que actúan como intermediarios, introduciendo latencia y comportamientos inconsistentes entre versiones del SO.

El último requisito puede funcionar en cualquiera de las 3 opciones. La única diferencia es que Nativo tiene APIs específicas para foldables (ventanas múltiples, modos de pantalla), mientras que Flutter y RN requieren más trabajo manual.

5. **Pregunta final:**

- **¿Qué es un aplicativo nativo, hay varios, como se clasifican?**

Un aplicativo nativo es aquel que se desarrolla específicamente para un sistema operativo, usando el lenguaje y las herramientas que ese SO reconoce de forma directa.

Se clasifican dependiendo del sistema operativo para el que fueron creados:

- Android: desarrollados con Java o Kotlin
- iOS: desarrollados con Swift u Objective-C
- Desktop: como aplicaciones para Windows, macOS o Linux

Referencias:

Google. (2026). [*Infografía sobre la evolución arquitectónica del desarrollo móvil*] [Imagen generada por IA]. Gemini. https://gemini.google.com